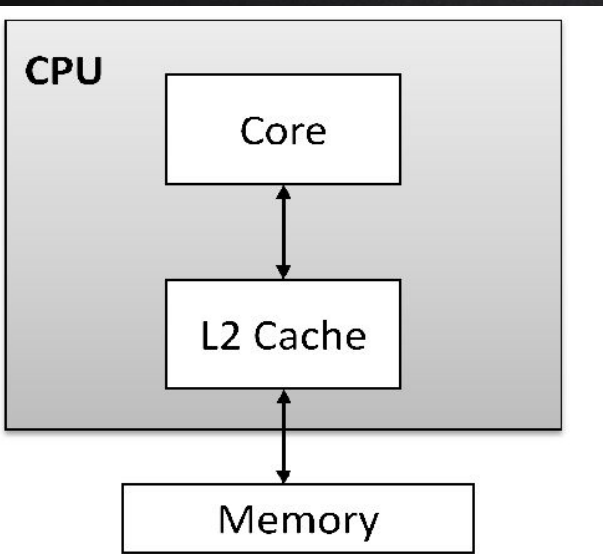


PROGRAMMATION ASYNCHRONE

LES PROCESSEURS

Processeurs avec 1 seul cœur



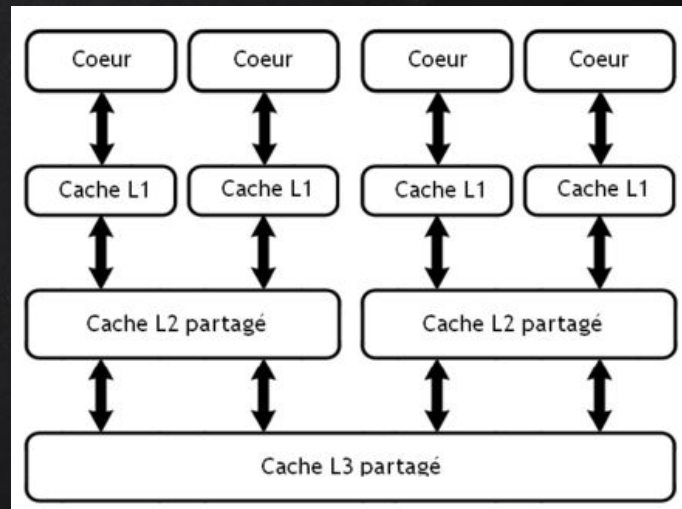
Concurrence mais pas de
parallélisme de tâche

SOLUTION

Respecter la loi de
MOORE (Un doublement
tous les 18 mois des
transistors) ? Oui

Augmenter la
fréquence? NON
Après les 5GHz effets
parasites
(effets quantiques,
désintégrations alpha)

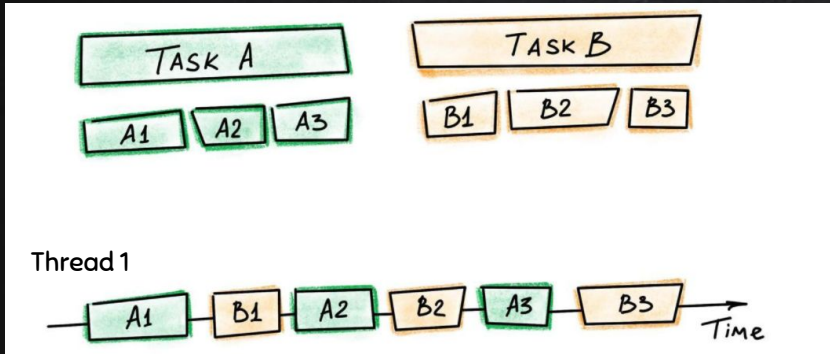
Processeurs multi-cœur



Concurrence et parallélisme
des tâches

CONCURRENCE / PARALLÈLE

Concurrency

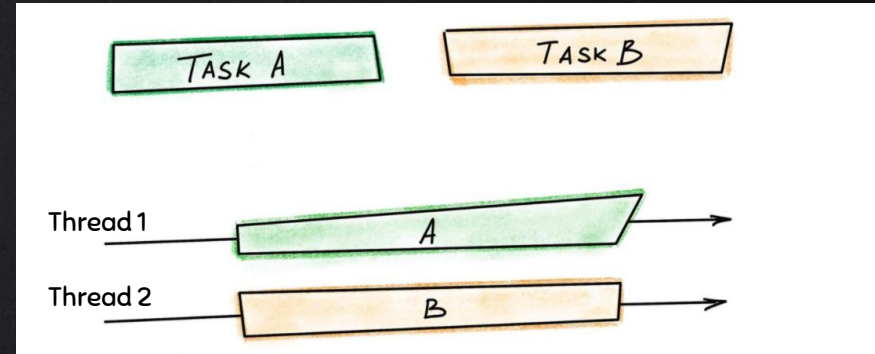


La concurrency est l'exécution de plusieurs tâches en cours de traitement dans des périodes de temps qui se chevauchent. Un détail important est que les tâches ne sont pas nécessairement effectuées en même temps (mais c'est possible).

Parallelism

Thread 1 exécuté sur le CPU 1

Thread 2 exécuté sur le CPU 2



Le parallélisme est littéralement l'exécution simultanée de tâches.

Le parallélisme est une sous-classe de la concurrence avant d'effectuer plusieurs tâches simultanées, vous devez d'abord les organiser correctement.

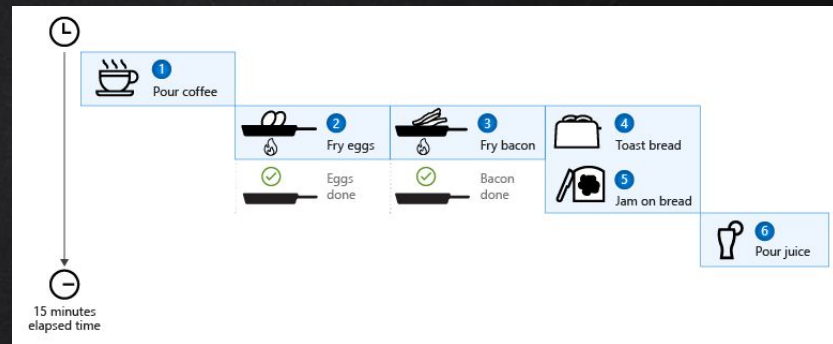
SYNCHRON / ASYNCHRON

Appel synchrone

Le flux de d'execution est linéaire une chose après l'autre.



Appel asynchrone

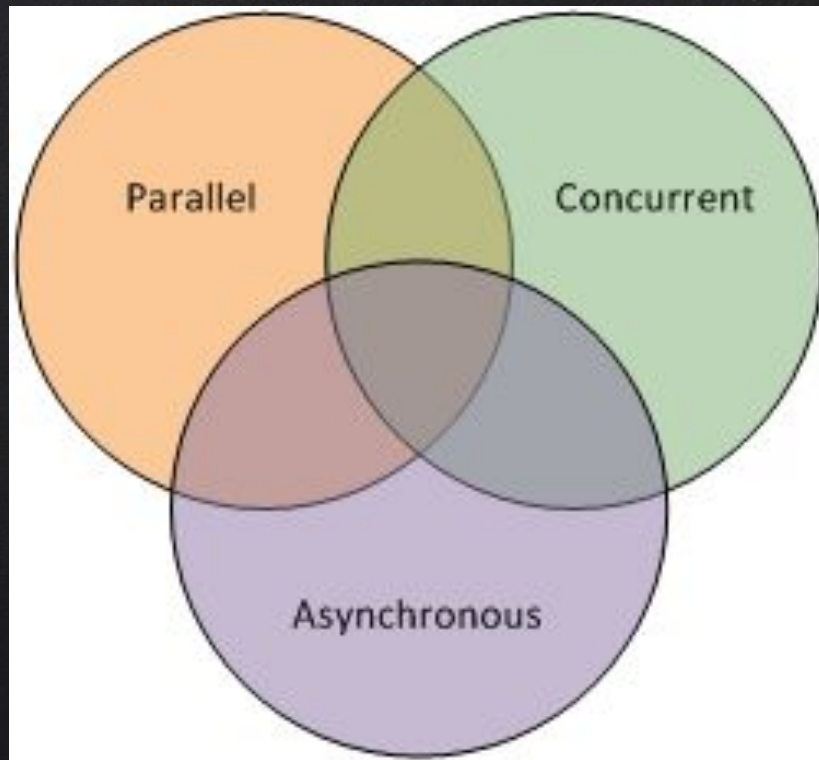


Le flux d'exécution est non linéaire, des tâches ont été lancées en arrière plan.

Cela évite de bloquer le thread principal !

Le lancement d'une tâche en arrière plan ne nécessite pas forcément un nouveau thread ! (Il est possible que ce soit un événement !)

PARALLEL $\neq$ CONCURRENT $\neq$ ASYNCHRONOUS



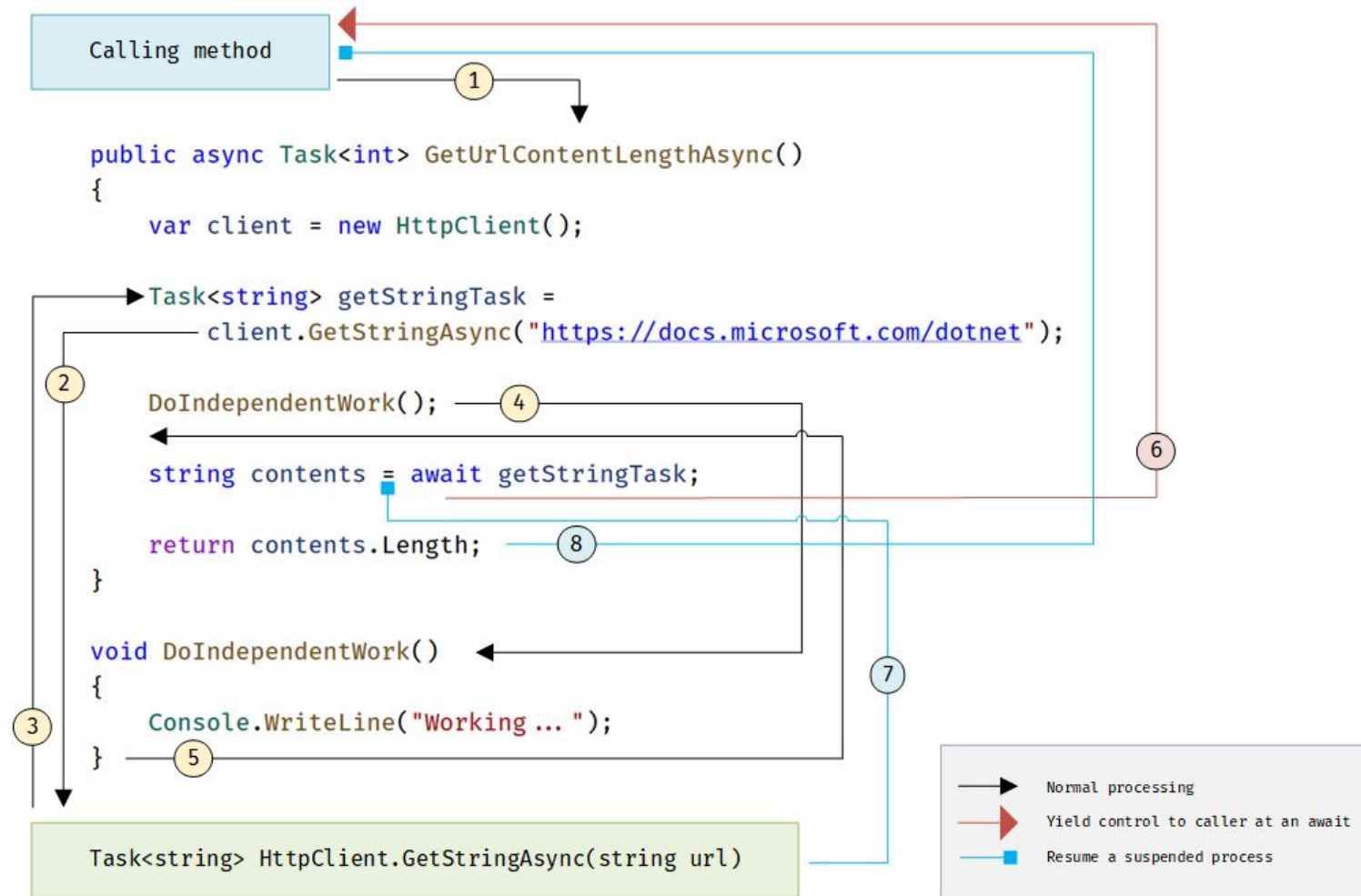
LA LIBRAIRIE TASK MODÈLE TAP

Les fonctions asynchrones :

- Si elle utilise le mot clé `await` la fonction doit comporter le mot clé `async` dans sa signature.
- Elle ont toujours le nom de la méthode suivi de `Async` par convention

```
public async Task NomDeLaMéthodeAsync() {  
    //Code de la fonction  
}
```

- Elle renvoie toujours un objet de type : `Task`, `Task<TResult>`



LA LIBRAIRIE TASK MODÈLE TAP

Lancement d'une tâche dans un nouveau thread :

`Task.Run(() => { //code à exécuter dans un thread });` (A éviter sur [ASP.net!](#))

Rendre une méthode asynchrone avec du code synchrone:

- Retourner une valeur : `Task.FromResult(//Valeur)`
- Retourner un void : `Task.CompletedTask`
- Lever une exception: `Task.FromException(//Exception)`

Attendre la fin d'une ou plusieurs tâches :

- `await tache1;`
- `await Task.WhenAny(//liste de tâches);`
- `await Task.WhenAll(//liste de tâches);`

Élément à utiliser...	Lorsque vous souhaitez effectuer cette opération...
<code>await</code>	Extraire le résultat d'une tâche en arrière-plan
<code>await Task.WhenAny</code>	Attendre la fin d'une tâche
<code>await Task.WhenAll</code>	Attendre la fin de toutes les tâches
<code>await Task.Delay</code>	Attendre pendant une période

TP 1

